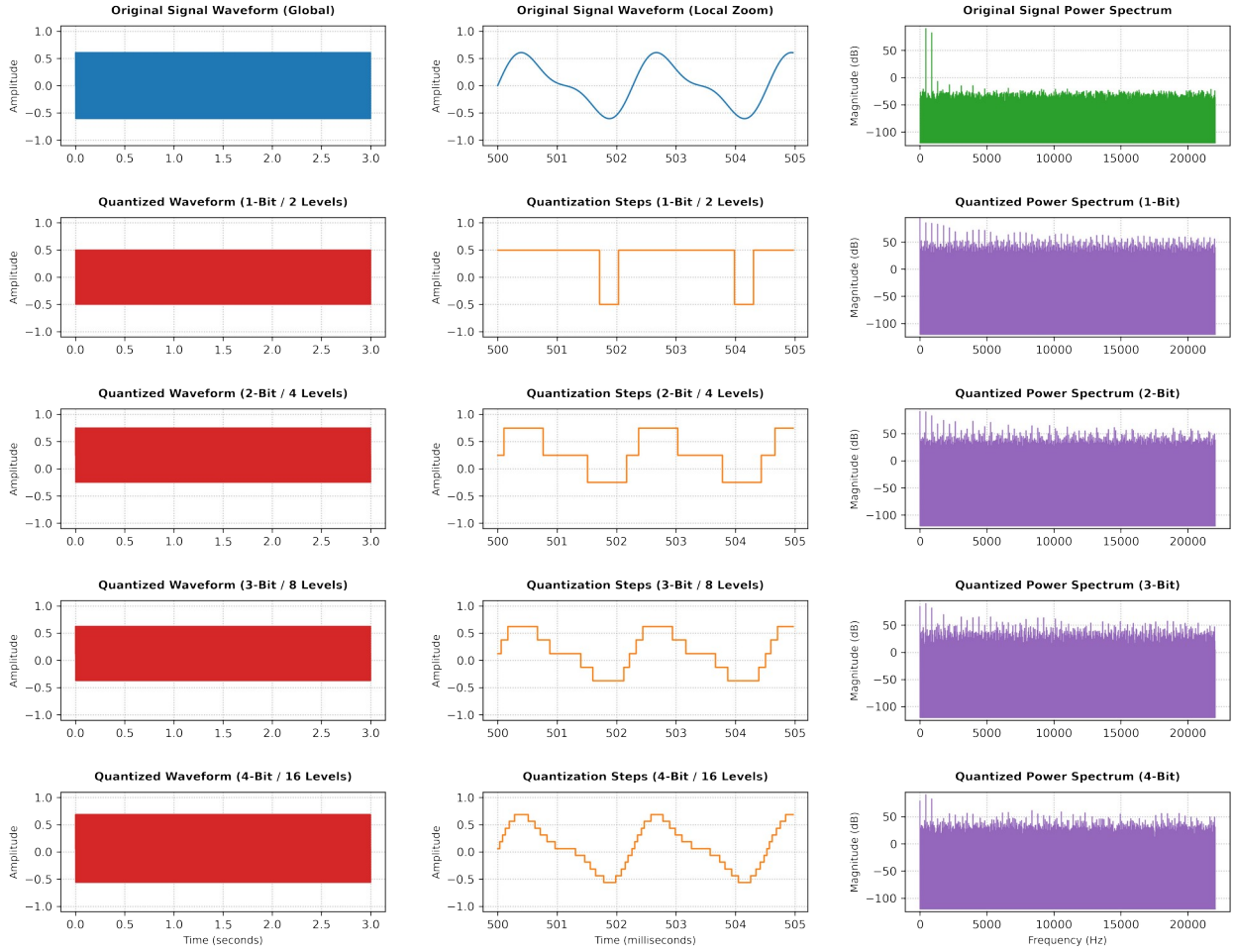
1.1 Uniform Quantization of Audio Perform uniform quantization on a short music file of the student's choice.Vary quantization levels (e.g., 2, 4, 8, 16 levels or corresponding bits).Listen to the original and quantized audio for subjective quality assessment.Observe the effect of quantization on the waveform and spectrum.

```
import wave
import numpy as np
import matplotlib.pyplot as plt
import time
import os
import platform

if platform.system() == "Windows":
    import winsound

sample_rate = 44100
t = np.linspace(0, 3, sample_rate * 3, endpoint=False)
dummy_signal = 0.5 * np.sin(2 * np.pi * 440 * t) + 0.2 * np.sin(2 *
np.pi * 880 * t)

signal_int16 = (dummy_signal * 32768).astype(np.int16)
with wave.open('test_audio.wav', 'wb') as wav_file:
    wav_file.setnchannels(1)
    wav_file.setsampwidth(2)
    wav_file.setframerate(sample_rate)
    wav_file.writeframes(signal_int16.tobytes())

with wave.open('test_audio.wav', 'rb') as wav_file:
    sample_rate = wav_file.getframerate()
    num_frames = wav_file.getnframes()
    raw_data = wav_file.readframes(num_frames)
    data = np.frombuffer(raw_data, dtype=np.int16)

data = data.astype(np.float32) / 32768.0

num_samples = min(len(data), 3 * sample_rate)
signal = data[:num_samples]
time_axis = np.arange(num_samples) / sample_rate

bit_depths = [1, 2, 3, 4]
num_plots = len(bit_depths) + 1

fig, axes = plt.subplots(num_plots, 3, figsize=(18, 2.8 * num_plots),
dpi=300)
plt.subplots_adjust(hspace=0.6, wspace=0.3)

axes[0, 0].plot(time_axis, signal, color='#1f77b4', linewidth=0.8)
axes[0, 0].set_title("Original Signal Waveform (Global)", fontsize=10,
fontweight='bold', pad=10)
axes[0, 0].set_ylabel("Amplitude", fontsize=9)
```

```
axes[0, 0].set_ylim(-1.1, 1.1)
axes[0, 0].grid(True, which='both', linestyle=':', alpha=0.5,
color='gray')

zoom_start = int(0.5 * sample_rate)
zoom_end = zoom_start + int(0.005 * sample_rate)

axes[0, 1].plot(time_axis[zoom_start:zoom_end] * 1000,
signal[zoom_start:zoom_end], color='#1f77b4', linewidth=1.2)
axes[0, 1].set_title("Original Signal Waveform (Local Zoom)",
fontsize=10, fontweight='bold', pad=10)
axes[0, 1].set_ylabel("Amplitude", fontsize=9)
axes[0, 1].set_ylim(-1.1, 1.1)
axes[0, 1].grid(True, which='both', linestyle=':', alpha=0.5,
color='gray')

fft_orig = np.abs(np.fft.rfft(signal))
freqs = np.fft.rfftfreq(num_samples, 1 / sample_rate)
fft_orig_db = 20 * np.log10(fft_orig + 1e-6)
axes[0, 2].plot(freqs, fft_orig_db, color='#2ca02c', linewidth=0.8)
axes[0, 2].set_title("Original Signal Power Spectrum", fontsize=10,
fontweight='bold', pad=10)
axes[0, 2].set_ylabel("Magnitude (dB)", fontsize=9)
axes[0, 2].set_ylim(np.min(fft_orig_db) - 5, np.max(fft_orig_db) + 5)
axes[0, 2].grid(True, which='both', linestyle=':', alpha=0.5,
color='gray')

print("Playing Track: Original Audio")
if platform.system() == "Windows":
    winsound.PlaySound('test_audio.wav', winsound.SND_FILENAME)
elif platform.system() == "Darwin":
    os.system('afplay test_audio.wav')
else:
    os.system('aplay test_audio.wav')
time.sleep(0.4)

for i, bits in enumerate(bit_depths):
    levels = 2**bits

    step_size = 2.0 / levels
    quantized_indices = np.round((signal - (-1.0)) / step_size)
    quantized_indices = np.clip(quantized_indices, 0, levels - 1)
    quantized = -1.0 + (quantized_indices * step_size) + (step_size /
2.0)

    out_signal = np.clip(quantized * 32768.0, -32768.0,
32767.0).astype(np.int16)
    filename = f"quantized_{bits}bit_{levels}levels.wav"
    with wave.open(filename, 'wb') as wav_file:
        wav_file.setnchannels(1)
```

```
        wav_file.setsampwidth(2)
        wav_file.setframerate(sample_rate)
        wav_file.writeframes(out_signal.tobytes())

    axes[i+1, 0].plot(time_axis, quantized, color='#d62728',
linewidth=0.8)
    axes[i+1, 0].set_title(f"Quantized Waveform ({bits}-Bit / {levels}
Levels)", fontsize=10, fontweight='bold', pad=10)
    axes[i+1, 0].set_ylabel("Amplitude", fontsize=9)
    axes[i+1, 0].set_ylim(-1.1, 1.1)
    axes[i+1, 0].grid(True, which='both', linestyle=':', alpha=0.5,
color='gray')

    axes[i+1, 1].step(time_axis[zoom_start:zoom_end] * 1000,
quantized[zoom_start:zoom_end], where='mid', color='#ff7f0e',
linewidth=1.2)
    axes[i+1, 1].set_title(f"Quantization Steps ({bits}-Bit / {levels}
Levels)", fontsize=10, fontweight='bold', pad=10)
    axes[i+1, 1].set_ylabel("Amplitude", fontsize=9)
    axes[i+1, 1].set_ylim(-1.1, 1.1)
    axes[i+1, 1].grid(True, which='both', linestyle=':', alpha=0.5,
color='gray')

    fft_quant = np.abs(np.fft.rfft(quantized))
    fft_quant_db = 20 * np.log10(fft_quant + 1e-6)
    axes[i+1, 2].plot(freqs, fft_quant_db, color='#9467bd',
linewidth=0.8)
    axes[i+1, 2].set_title(f"Quantized Power Spectrum ({bits}-Bit)",
fontsize=10, fontweight='bold', pad=10)
    axes[i+1, 2].set_ylabel("Magnitude (dB)", fontsize=9)
    axes[i+1, 2].set_ylim(np.min(fft_orig_db) - 5, np.max(fft_orig_db)
+ 5)
    axes[i+1, 2].grid(True, which='both', linestyle=':', alpha=0.5,
color='gray')

    print(f"Playing Track: {bits}-Bit Quantized Audio ({levels}
Levels)")
    if platform.system() == "Windows":
        winsound.PlaySound(filename, winsound.SND_FILENAME)
    elif platform.system() == "Darwin":
        os.system(f'afplay {filename}')
    else:
        os.system(f'aplay {filename}')
    time.sleep(0.4)

axes[-1, 0].set_xlabel("Time (seconds)", fontsize=9)
axes[-1, 1].set_xlabel("Time (milliseconds)", fontsize=9)
axes[-1, 2].set_xlabel("Frequency (Hz)", fontsize=9)

plt.savefig('academic_quantization_report.png', bbox_inches='tight',
```

```
dpi=300)
plt.show()

Playing Track: Original Audio
Playing Track: 1-Bit Quantized Audio (2 Levels)
Playing Track: 2-Bit Quantized Audio (4 Levels)
Playing Track: 3-Bit Quantized Audio (8 Levels)
Playing Track: 4-Bit Quantized Audio (16 Levels)
```



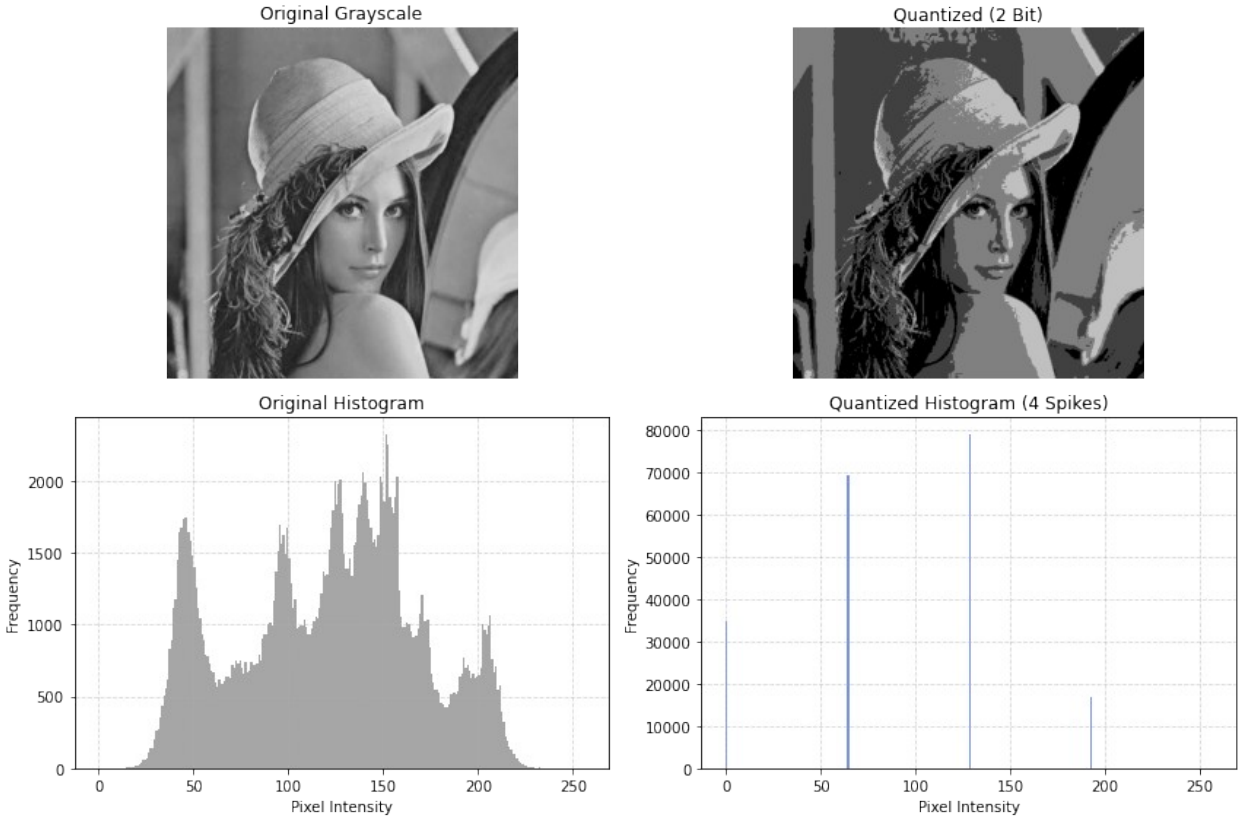
1.2 Uniform Quantization of a Grayscale Image Perform uniform quantization on a grayscale image of the student's choice.If starting from a color image, convert to grayscale first.Apply uniform quantization at different bit depths (e.g., 2-bit, 4-bit, 8-bit). Display original and quantized images side-by-side.Plot histograms for original and quantized images.Evaluate performance using MSE and SNR between original and quantized images.

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

image_path = "lena_colored.jpg"
```

```
levels = 4

img_gray = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

step_size = 256 / levels

bin_indices = np.floor(img_gray / step_size)

quantized_img = (bin_indices * step_size).astype(np.uint8)

error = img_gray.astype(np.float64) - quantized_img.astype(np.float64)
mse = np.mean(error**2)

signal_power = np.mean(img_gray.astype(np.float64) ** 2)

if mse == 0:
    snr = float("inf")
else:
    snr = 10 * np.log10(signal_power / mse)

print("--- Evaluation Metrics ---")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Signal-to-Noise Ratio (SNR): {snr:.4f} dB")

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(img_gray, cmap="gray", vmin=0, vmax=255)
plt.title("Original Grayscale")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(quantized_img, cmap="gray", vmin=0, vmax=255)
plt.title(f"Quantized (2 Bit)")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.hist(img_gray.ravel(), bins=256, range=(0, 256), color="gray",
alpha=0.7)
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.grid(True, linestyle="--", alpha=0.5)

plt.subplot(2, 2, 4)
plt.hist(
    quantized_img.ravel(),
    bins=256,
    range=(0, 256),
    color="royalblue",
```

```
    alpha=0.7,
)
plt.title(f"Quantized Histogram ({levels} Spikes)")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()
```

```
--- Evaluation Metrics ---
Mean Squared Error (MSE): 1319.9967
Signal-to-Noise Ratio (SNR): 11.0512 dB
```

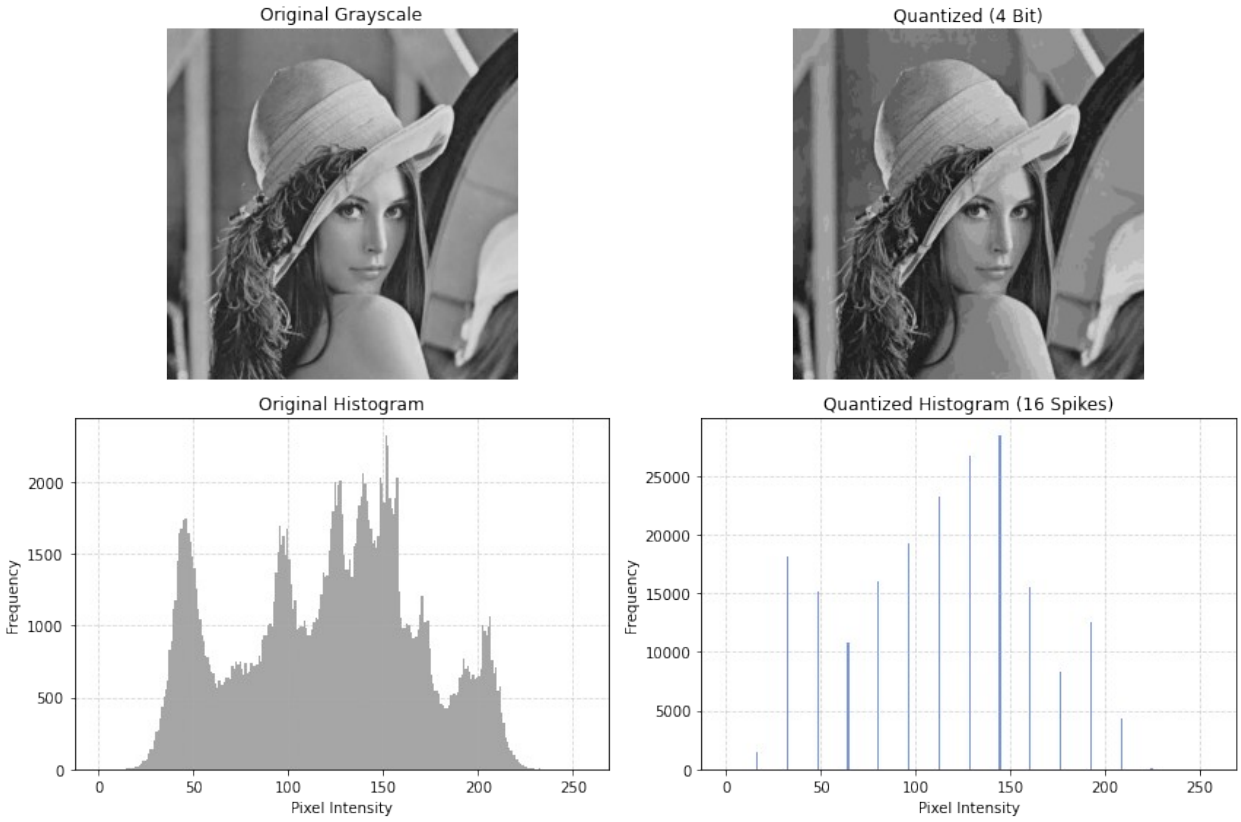



```
import cv2
import matplotlib.pyplot as plt
import numpy as np

image_path = "lena_colored.jpg"
levels = 16

img_gray = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

step_size = 256 / levels
```

```
bin_indices = np.floor(img_gray / step_size)

quantized_img = (bin_indices * step_size).astype(np.uint8)

error = img_gray.astype(np.float64) - quantized_img.astype(np.float64)
mse = np.mean(error**2)

signal_power = np.mean(img_gray.astype(np.float64) ** 2)

if mse == 0:
    snr = float("inf")
else:
    snr = 10 * np.log10(signal_power / mse)

print("--- Evaluation Metrics ---")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Signal-to-Noise Ratio (SNR): {snr:.4f} dB")

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(img_gray, cmap="gray", vmin=0, vmax=255)
plt.title("Original Grayscale")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(quantized_img, cmap="gray", vmin=0, vmax=255)
plt.title(f"Quantized (4 Bit)")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.hist(img_gray.ravel(), bins=256, range=(0, 256), color="gray",
alpha=0.7)
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.grid(True, linestyle="--", alpha=0.5)

plt.subplot(2, 2, 4)
plt.hist(
    quantized_img.ravel(),
    bins=256,
    range=(0, 256),
    color="royalblue",
    alpha=0.7,
)
plt.title(f"Quantized Histogram ({levels} Spikes)")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
```

```
plt.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()
```

```
--- Evaluation Metrics ---
Mean Squared Error (MSE): 81.2422
Signal-to-Noise Ratio (SNR): 23.1591 dB
```




```
import cv2
import matplotlib.pyplot as plt
import numpy as np

image_path = "lena_colored.jpg"
levels = 256

img_gray = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

step_size = 256 / levels

bin_indices = np.floor(img_gray / step_size)

quantized_img = (bin_indices * step_size).astype(np.uint8)

error = img_gray.astype(np.float64) - quantized_img.astype(np.float64)
```

```
mse = np.mean(error**2)

signal_power = np.mean(img_gray.astype(np.float64) ** 2)

if mse == 0:
    snr = float("inf")
else:
    snr = 10 * np.log10(signal_power / mse)

print("--- Evaluation Metrics ---")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Signal-to-Noise Ratio (SNR): {snr:.4f} dB")

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(img_gray, cmap="gray", vmin=0, vmax=255)
plt.title("Original Grayscale")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(quantized_img, cmap="gray", vmin=0, vmax=255)
plt.title(f"Quantized (8 Bit)")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.hist(img_gray.ravel(), bins=256, range=(0, 256), color="gray",
alpha=0.7)
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.grid(True, linestyle="--", alpha=0.5)

plt.subplot(2, 2, 4)
plt.hist(
    quantized_img.ravel(),
    bins=256,
    range=(0, 256),
    color="royalblue",
    alpha=0.7,
)
plt.title(f"Quantized Histogram ({levels} Spikes)")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()
```

```
--- Evaluation Metrics ---
Mean Squared Error (MSE): 0.0000
Signal-to-Noise Ratio (SNR): inf dB
```